

Section 6.2: Distributed Consensus, ID Generation & Service Discovery

Part of a 3-File Distributed Systems Series (File 2 of 3)

Study Note: This section covers consensus algorithms with significant mathematical underpinning. Extra weight is given to worked numeric examples, quorum arithmetic proofs, and proof-by-contradiction correctness arguments. For every algorithm, an adversarial failure scenario is walked through step by step.

1. Leader Election & Consensus Algorithms

Why Consensus is Hard — FLP Impossibility (Proof Sketch)

Before diving into algorithms, understand *why* this is a hard problem.

FLP Impossibility Theorem (Fischer, Lynch, Paterson 1985): > In a fully asynchronous distributed system, no deterministic consensus algorithm can guarantee termination if even *one* process may fail.

Proof by contradiction (simplified): - Assume algorithm A guarantees consensus with one possible faulty process. - Construct a “bivalent” initial configuration: the system could decide either 0 or 1. - Show that for every step A takes, an adversary can delay messages such that the system remains bivalent (undecided) indefinitely. - Therefore A never terminates in the adversary’s schedule. Contradiction.

Practical resolution: Real algorithms (Raft, Paxos) escape FLP by assuming *partial synchrony* — eventually, messages arrive within some unknown but finite bound. They trade guaranteed termination for *probabilistic* termination under realistic network conditions.

1.1 Paxos

Three Roles: - **Proposer:** Initiates proposals - **Acceptor:** Votes on proposals
- **Learner:** Learns the decided value

Two-Phase Protocol (Single Instance):

Phase 1a – Prepare:
Proposer picks proposal number n (must be $>$ any n it has used)
Sends: PREPARE(n) to majority of Acceptors

Phase 1b – Promise:
Acceptor receives PREPARE(n):
If $n > \text{highest_promised}$:
Update $\text{highest_promised} = n$
Reply: PROMISE(n , $\text{highest_accepted_n}$, $\text{highest_accepted_v}$)
Else:
Ignore (or reply NACK)

Phase 2a – Accept:
Proposer receives promises from majority:
If any promise included (n' , v') with highest n' :
Must propose $v = v'$ (cannot change an already-chosen value!)
Else:
Can propose any value v
Sends: ACCEPT(n , v) to majority of Acceptors

Phase 2b – Accepted:
Acceptor receives ACCEPT(n , v):
If $n \geq \text{highest_promised}$:
Accept the value
Reply: ACCEPTED(n , v) to Proposer + all Learners
Else:
Ignore

Decision:
When Learner receives ACCEPTED(n , v) from majority $\rightarrow$ value v is chosen.

Worked Numeric Example — Paxos with Competing Proposers:

Cluster: 3 Acceptors (A1, A2, A3). Quorum = 2.

Round 1: Proposer P1 proposes value "blue" with $n=1$
P1 $\rightarrow$ A1, A2: PREPARE(1)
A1, A2: PROMISE(1, null, null) [no prior accepts]
P1 $\rightarrow$ A1, A2: ACCEPT(1, "blue")
A1: ACCEPTED(1, "blue")
[Network partition: A2's ACCEPTED message lost]
P1 never hears from quorum $\rightarrow$ Round 1 fails

Round 2: Proposer P2 proposes "red" with $n=2$
P2 $\rightarrow$ A1, A3: PREPARE(2)
A1: PROMISE(2, $\text{accepted_n}=1$, $\text{accepted_v}=\text{"blue"}$) $\leftarrow$ key!
A3: PROMISE(2, null, null)
P2 sees highest accepted = ($n=1$, $v=\text{"blue"}$) $\rightarrow$ MUST use "blue"
P2 $\rightarrow$ A1, A3: ACCEPT(2, "blue") [forced to use "blue", not "red"]
A1, A3: ACCEPTED(2, "blue")

Decision: "blue" chosen ✓

Safety proof: P2 was forced to continue "blue" because A1 had already accepted it. This prevents two different values from being chosen.

Why Multi-Paxos skips Phase 1:

Once a stable leader is elected, it has already received promises from a quorum. It can skip PREPARE for subsequent rounds and go straight to ACCEPT. This reduces 2 network round-trips to 1 per consensus decision — a dramatic performance improvement for high-throughput systems.

1.2 Raft

Raft was designed to be *understandable* — it achieves the same safety as Paxos but with clearly separated concerns.

Three Server States:

Follower → Candidate → Leader
↑
(loses election or hears from leader)

Leader Election — Detailed Flow:

All nodes start as Followers.

Election timeout: randomly chosen 150–300ms per node.

Step 1: Follower F times out (no heartbeat from leader)

F increments its term: $\text{term} = T+1$

F transitions to Candidate

F votes for itself

F sends RequestVote($\text{term}=T+1$, $\text{candidateId}=F$, lastLogIndex , lastLogTerm) to all

Step 2: Other nodes receive RequestVote

Grant vote if ALL of:

(a) $\text{term} \geq \text{their current term}$

(b) They haven't voted in this term yet

(c) Candidate's log is at least as up-to-date as theirs

($\text{lastLogTerm} > \text{myLastTerm}$, OR $\text{lastLogTerm} == \text{myLastTerm}$ AND $\text{lastLogIndex} \geq \text{myLastIndex}$)

Step 3: F receives votes from majority → becomes Leader

Immediately sends heartbeat AppendEntries(empty) to all followers

Followers reset their election timers

Split vote scenario:

Two candidates C1, C2 each get 2 of 5 votes (neither gets majority of 3)

Both wait out their random timeout and start new election in term T+2
Random timeouts make split votes statistically unlikely to repeat

Worked Numeric Example — Log Replication:

5-node cluster: Leader L, Followers F1-F4
Quorum = 3

Client sends: "SET x=10"

L appends to local log: [index=5, term=3, cmd="SET x=10"]

L sends AppendEntries to F1, F2, F3, F4

Responses:

F1: ACK (log index 5 committed locally)

F2: ACK

F3: [slow, no response yet]

F4: [partitioned, no response]

L has received ACKs from: itself + F1 + F2 = 3 nodes = quorum ✓

L marks entry as committed

L responds to client: "OK"

Later: F3 reconnects, catches up via AppendEntries

F4 reconnects, L sends missing entries starting from F4's last known index

Why Raft is Safe During Leader Crash (proof by contradiction):

Assume a new leader L2 is elected that does NOT have a committed entry E.

- E was committed means it was replicated to a majority M1.
- L2 was elected means it received votes from a majority M2.
- Any two majorities overlap: $M1 \cap M2 \neq \emptyset$.
- The overlapping node voted for L2, so L2's log must be at least as up-to-date as that node's.
- That node has E in its log.
- Therefore L2's log must include E. Contradiction.

Conclusion: A Raft leader always has all committed entries. Safety guaranteed.

1.3 ZooKeeper Atomic Broadcast (Zab)

Zab is Paxos-family but optimized for ZooKeeper's primary-backup model.

Key difference from Raft:

- Zab uses epoch numbers (like terms) + zxid (transaction ID)
- $zxid = (epoch \ll 32) \mid counter$ — 64-bit monotonic ID
- Leader commits only when quorum ACKs the PROPOSAL

Write flow:

Client → Any node → Forwarded to Leader
Leader: PROPOSAL(zxid, data) → All Followers
Followers ACK → Leader gets quorum ACKs → COMMIT(zxid)
Leader → All Followers: COMMIT
Followers apply to state machine

Read flow (important!):

Reads can be served by ANY node (including followers)
→ Possible stale reads (eventual within ZooKeeper's model)
→ For linearizable reads, client must call sync() first

Split-Brain Resolution — Fencing Tokens (Worked Example):

Scenario: Leader L1 pauses (GC stop-the-world for 10 seconds).
Cluster elects new leader L2.
L1 resumes, thinks it's still leader.

Without fencing:

L1 writes to storage with stale authority → corruption

With fencing tokens:

L1 was issued token=42 on election
L2 is issued token=43 on election

L1 tries to write: storage.write(data, fence_token=42)
Storage checks: 42 < current_token (43) → REJECT

L2 writes: storage.write(data, fence_token=43)
Storage checks: 43 == current_token → ACCEPT ✓

Implementation: ZooKeeper epoch number serves as fencing token.
External storage (HDFS, S3) must check and enforce the token.

1.4 Algorithm Comparison: When to Use Which

Dimension	Paxos	Raft	Zab
Understandability	Low (complex)	High (designed for clarity)	Medium
Leader election	Implicit	Explicit, randomized timeout	Explicit, priority-based
Log ordering	Per-instance	Sequential log index	zxid (epoch + counter)
Leader change cost	Phase 1 replay	Log catch-up	Epoch sync

Dimension	Paxos	Raft	Zab
Read scalability	Low (leader reads)	Low (leader reads by default)	High (follower reads)
Best for	Academic foundation, Chubby	General distributed systems, etcd, CockroachDB	ZooKeeper coordination
Production examples	Google Chubby	etcd, CockroachDB, TiKV	Apache ZooKeeper

Design use case guidance: - Building a distributed key-value store from scratch → **Raft** (well-documented, clear correctness proofs) - Need high read throughput for configuration/coordination → **ZooKeeper (Zab)** (follower reads) - Operating in a system already using Paxos infrastructure → **Multi-Paxos** (skip Phase 1 after leader stable)

2. Quorum Parameter Engineering

The Fundamental Formula

For strong consistency: **$W + R > N$**

- W = write acknowledgment count required
- R = read replica count required
- N = total replica count

Why $W + R > N$ guarantees consistency (proof):

If $W + R > N$, then any write quorum and any read quorum must overlap by at least one node. That overlapping node has the latest write. The read will always see it.

Formally: $|W_set \cap R_set| = |W_set| + |R_set| - |W_set \cup R_set| \geq W + R - N > 0$.

Worked Numeric Scenarios

Scenario A: $N=5, W=3, R=3$

$W + R = 6 > 5$ ✓ Strong consistency guaranteed
 Overlap = $W + R - N = 1$ node minimum
 Write survives: up to $N - W = 2$ node failures
 Read survives: up to $N - R = 2$ node failures

Latency: Must wait for 3rd slowest of 5 nodes to respond
Availability: Survives 2 simultaneous failures

Scenario B: N=5, W=1, R=1

$W + R = 2 \leq 5$ ✗ No strong consistency guarantee
Can miss latest write (write on node A, read from node B)

Write latency: Fastest node only (~1ms)
Available during: Any single node alive
Use for: Metrics, analytics, non-critical writes

Scenario C: N=5, W=5, R=1

$W + R = 6 > 5$ ✓ Strong consistency
All replicas must confirm write → W is bottleneck

Write latency: Slowest of all 5 nodes (tail latency)
Write availability: All 5 nodes must be up (brittle!)
Read latency: Fastest node (~1ms)

Anti-pattern: Almost never correct. One node failure blocks all writes.

Scenario D: N=3, W=2, R=2 (Dynamo default for strong)

$W + R = 4 > 3$ ✓
Overlap: 1 node minimum
Survives: 1 failure for reads, 1 failure for writes

This is the standard "Dynamo quorum" – balanced, practical.

Tuning for Different Workloads

Read-heavy system (10:1 read/write ratio):

Prefer W=3, R=1 of N=3
 $W + R = 4 > 3$ ✓ Still consistent
Reads are single-node: very fast
Writes hit all replicas: slower, but rare

Write-heavy system (1:10 read/write ratio):

Prefer W=1, R=3 of N=3 (or give up strong consistency)
 $W + R = 4 > 3$ ✓ Still consistent
Writes are single-node: very fast
Reads hit all replicas: slower, but rare

Latency-sensitive, consistency optional:

W=1, R=1 with async anti-entropy
Accept stale reads; resolve conflicts with vector clocks

3. Distributed ID Generation

Why Distributed IDs Are Hard

A centralized auto-increment counter is a single point of failure and a bottleneck. Distributed ID generation must provide uniqueness *without coordination* (or with minimal coordination).

3.1 Snowflake Algorithm (Twitter)

64-bit Layout:

Bit 63 : Sign bit (always 0)
Bits 22-62: Timestamp (41 bits) – milliseconds since custom epoch
Bits 17-21: Data center ID (5 bits) – up to 32 DCs
Bits 12-16: Machine ID (5 bits) – up to 32 machines per DC
Bits 0-11 : Sequence (12 bits) – up to 4096 IDs per millisecond

Total unique IDs per millisecond per machine: 4,096
Total machines: $32 \times 32 = 1,024$
Total IDs per millisecond globally: $4,096 \times 1,024 \approx 4.2$ million
Total capacity before timestamp exhausts: 2^{41} ms ≈ 69 years

Worked Example — ID Decoding:

ID: 7,326,403,215,965,184,000 (decimal)

Binary: 0 11001011 00000000 00000000 10000 00001 000000000001

Timestamp bits (41): 11001011... = 1,686,000,000 ms since Twitter epoch
= Unix time 1,686,000,000 + epoch_offset

Data center (5): 10000 = DC 16

Machine (5): 00001 = Machine 1

Sequence (12): 000000000001 = 1 (second ID in this millisecond)

Insight: IDs are time-sortable. ORDER BY id $\approx$ ORDER BY created_at.

Clock Drift Problem and Solution:

Problem: NTP adjusts clock backward by 5ms.
Generator sees timestamp T=1000ms, then next call T=995ms.
Using T=995ms would generate duplicate IDs!

Solution:

```
def next_id(self):
    ts = current_ms()
    if ts < self.last_ts:
        drift = self.last_ts - ts
        if drift > MAX_DRIFT_MS (e.g., 10ms):
            raise ClockDriftError(f"Clock moved back {drift}ms")
        else:
            # Small drift: wait until caught up
            time.sleep(drift / 1000)
            ts = current_ms()

    if ts == self.last_ts:
        self.seq = (self.seq + 1) & 0xFFF # 12-bit mask
        if self.seq == 0:
            # Sequence exhausted in this ms – wait for next ms
            ts = self._spin_until_next_ms()
    else:
        self.seq = 0

    self.last_ts = ts
    return (ts << 22) | (self.dc_id << 17) | (self.machine_id << 12) | self.seq
```

3.2 UUID Versions Compared

UUID v4 (Random):

128 random bits (with 6 bits used for version/variant)
Effective randomness: 122 bits
Collision probability for 1 billion UUIDs: $\sim 6 \times 10^{-10}$ (negligible)

Problem: Not sortable. Database B-tree index has random insertion points
→ Page splits, cache misses, 3-10× slower INSERT than sequential IDs

UUID v7 (Time-ordered, RFC 9562):

Bits 0-47: Unix timestamp in milliseconds (48 bits)
Bits 48-51: Version = 7 (4 bits)
Bits 52-63: Random or sequence (12 bits)
Bits 64-65: Variant (2 bits)
Bits 66-127: Random (62 bits)

Benefits over v4:

- Lexicographically sortable (timestamp prefix)
- Database-friendly: sequential inserts, no page splits
- No coordination needed
- Still globally unique

Numeric comparison — DB insert performance:

PostgreSQL INSERT benchmark (1M rows):

UUID v4 (random): ~45,000 rows/sec

[Random index positions → frequent page splits → high I/O]

UUID v7 (time-ordered): ~95,000 rows/sec

[Sequential index positions → minimal page splits → low I/O]

Snowflake: ~100,000 rows/sec

[Also time-ordered, plus smaller (64-bit vs 128-bit)]

Auto-increment: ~110,000 rows/sec

[Optimal but requires coordination]

3.3 ULID vs. Snowflake vs. UUID — Full Comparison

Property	Snowflake	UUID v4	UUID v7	ULID
Size	64-bit	128-bit	128-bit	128-bit
Time-sortable	✓	✗	✓	✓
Coordination	Node ID assignment	None	None	None
Collision risk	Zero (by design)	Negligible	Negligible	Negligible
Info leakage	Timestamp + node	None	Timestamp	Timestamp
DB index perf	Excellent	Poor	Good	Good
Human readable	No	No	No	Yes (base32)
Best for	Internal microservices	External/public IDs	General purpose	Logs, events

Decision rule: - Internal service IDs, high throughput → Snowflake - Public-facing IDs (don't leak node info) → UUID v7 - Need URL-safe, human-readable → ULID - Legacy system expecting 128-bit UUIDs → UUID v7 (drop-in for v4 with sorting benefit)

4. Service Discovery Patterns

4.1 Client-Side vs. Server-Side Discovery

Client-Side Discovery:

Flow:

```
Service A starts → registers with Consul: {name: "payments", addr: "10.0.1.5:8080"}
Service B needs to call payments:
  B queries Consul: GET /v1/catalog/service/payments
  Consul returns: [{addr: "10.0.1.5:8080"}, {addr: "10.0.1.6:8080"}]
  B selects instance (round-robin, least-connections, etc.)
  B calls selected instance directly
```

Benefits:

- No extra network hop
- Client can implement sophisticated load balancing (latency-aware, zone-aware)

Drawbacks:

- Every service must implement discovery logic (polyglot complexity)
- Registry outage = all services lose discovery
- Client must handle stale registry entries

Server-Side Discovery:

Flow:

```
Service A registers with registry
Load balancer periodically polls registry, maintains routing table
Service B → Load Balancer → healthy instance of A (transparent to B)
```

Benefits:

- Zero client code changes
- Centralized routing policy
- Works for any protocol (TCP, gRPC, HTTP)

Drawbacks:

- LB is bottleneck and single point of failure (mitigate with LB HA)
- Extra network hop adds ~1ms latency
- LB must be per-protocol-aware for advanced routing

When to use which:

Scenario	Recommendation
Homogeneous service language (all Go, all Java)	Client-side: use language SDK
Polyglot microservices	Server-side: avoids per-language discovery impl

Scenario	Recommendation
Low-latency requirements (<5ms)	Client-side: remove extra hop
Kubernetes environment	Server-side (kube-proxy / ingress built-in)
Need circuit breaking, retries	Client-side with service mesh (Envoy sidecar)

4.2 Health Check Comparison

Method	Detection Speed	Reliability	Overhead
Push heartbeat (service → registry)	Fast (~30s)	Medium: process crash detected; but hanging process still sends	Low (service-initiated)
Pull probe (registry → service)	Slower (~60s)	High: actually verifies liveness	Medium (registry-initiated)
HTTP /health endpoint	Configurable	High: checks dependencies too	Low-Medium
TCP connect check	Configurable	Medium: port open ≠ app healthy	Very low

Best practice: Combine pull probe (TCP/HTTP) with push heartbeat. Pull detects zombie processes; push detects network partitions faster.

5. Interview Problems with Full Solutions

Problem 1: Design a Distributed ID Generator for 1,000 Nodes

Requirements: - Unique IDs across 1,000 nodes - Time-sortable - No central coordinator in hot path - Handle clock drift

Solution — Extended Snowflake:

Problem with standard Snowflake: 5+5=10 bits for node = 1,024 max nodes.
1,000 nodes fits, but barely. One bit flip from collision if misconfigured.

Better layout for 1,000 nodes:

64-bit ID:

Bit 63: Sign (0)

Bits 22-62: Timestamp (41 bits) – 69 years
Bits 12-21: Node ID (10 bits) – 1,024 nodes
Bits 0-11: Sequence (12 bits) – 4,096/ms/node

Node ID assignment options:

- A) ZooKeeper: Node registers, gets auto-increment node_id from ZK
Pros: Simple, reliable Cons: ZK dependency at startup
- B) MAC address hash: hash(MAC) mod 1024
Pros: No coordination Cons: Hash collisions possible
- C) Redis INCR at startup: node_id = INCR("node_counter") mod 1024
Pros: Fast Cons: Redis dependency, IDs wrap around

Recommended: ZooKeeper ephemeral node with sequential path.

/nodes/node-0000000042 → node_id = 42

Node crash: ephemeral node deleted → ID available for reuse

Clock drift handling:

```
class SnowflakeGenerator:
    MAX_SEQUENCE = 4095      # 12 bits
    MAX_DRIFT_MS = 10        # Alert threshold

    def __init__(self, node_id: int):
        assert 0 <= node_id < 1024
        self.node_id = node_id
        self.sequence = 0
        self.last_ts = -1

    def next_id(self) -> int:
        ts = self._ms()

        if ts < self.last_ts:
            drift = self.last_ts - ts
            if drift > self.MAX_DRIFT_MS:
                raise RuntimeError(f"Clock drifted back {drift}ms – ID generation
halted")
            # Small drift: wait it out safely
            time.sleep((drift + 1) / 1000)
            ts = self._ms()

        if ts == self.last_ts:
            self.sequence = (self.sequence + 1) & self.MAX_SEQUENCE
            if self.sequence == 0:
                # Exhausted 4096 IDs in 1ms – spin until next ms
                ts = self._spin_next_ms()
        else:
            self.sequence = 0

        self.last_ts = ts
        return (ts << 22) | (self.node_id << 12) | self.sequence

    def _ms(self) -> int:
        return int(time.time() * 1000)
```

```
def _spin_next_ms(self) -> int:
    ts = self._ms()
    while ts <= self.last_ts:
        ts = self._ms()
    return ts
```

Sequence overflow math:

At 4,096 IDs/ms/node × 1,000 nodes = 4,096,000 IDs/ms globally
= 4.096 billion IDs/second

If a single node needs >4,096 IDs/ms, it must wait ~1ms.
This is a natural throttle: ~4M writes/sec per node is well beyond any realistic DB write rate.
Sequence overflow in practice: essentially never.

Problem 2: Service Discovery for a Multi-Region Microservices Platform

Requirements: - 50 microservices, 3 regions (US, EU, APAC) - Services must prefer local-region instances (latency optimization) - Cross-region failover if local region is fully down - Health checks must detect both process crash and database unavailability

Architecture:

Layer 1 – Local registry: Consul per region
Each region has its own Consul cluster (3-5 nodes, Raft)
Services register to local Consul only

Layer 2 – Global federation: Consul WAN federation
Consul clusters share service catalog across regions
Tagged with region: {region: "us-east", zone: "us-east-1a"}

Layer 3 – Client with zone-aware routing:

1. Query local Consul for service instances
2. Filter: prefer same-region instances (tag: region=current)
3. If none healthy in region → fallback to WAN catalog
4. Load balance among selected instances

Health check implementation:

```
# Consul service registration with layered health checks
service:
  name: "payment-service"
  port: 8080
  tags: ["region:us-east", "version:2.1.4"]
  checks:
```

```
# Check 1: Process liveness (TCP)
- id: "tcp-alive"
  tcp: "localhost:8080"
  interval: "10s"
  timeout: "2s"

# Check 2: Application health (HTTP with dependency checks)
- id: "http-health"
  http: "http://localhost:8080/health/ready"
  interval: "15s"
  timeout: "5s"
  # /health/ready checks: DB connection, cache connection, downstream services

# Check 3: Push heartbeat (service sends to Consul)
- id: "heartbeat"
  ttl: "30s" # Service must call PUT /agent/check/pass/heartbeat every <30s
```

Failover logic (client-side):

```
def discover_service(service_name: str, current_region: str) -> str:
    # Step 1: Local region instances
    local = consul.health.service(service_name, tag=f"region:{current_region}",
    passing=True)
    if local:
        return load_balance(local) # Prefer local

    # Step 2: Cross-region fallback (adds ~80-160ms latency)
    all_regions = consul.health.service(service_name, passing=True)
    if all_regions:
        # Sort by region proximity (pre-configured distance matrix)
        ranked = sorted(all_regions, key=lambda s: REGION_DISTANCE[current_region]
        [s.region])
        return load_balance(ranked[:5]) # Top 5 nearest instances

    raise ServiceUnavailableError(f"{service_name} has no healthy instances
    globally")
```

Problem 3: Quorum Tuning for a Read-Heavy Configuration Store

Requirements: - Configuration data read ~100,000 times/sec - Written ~10 times/sec (rare operator updates) - Must be strongly consistent (stale config causes incidents) - 5-node cluster, 2 AZs (3 nodes in AZ-A, 2 in AZ-B)

Analysis:

```
Workload: 100,000 reads/sec, 10 writes/sec
Ratio: 10,000:1
```

Naive strong consistency: $W=3, R=3$

Every read hits 3 nodes $\rightarrow 300,000$ node-read-ops/sec (expensive)

Better: $W=4, R=2$ (still satisfies $W+R=6 > N=5$)

Every read hits 2 nodes $\rightarrow 200,000$ node-read-ops/sec

Every write hits 4 nodes $\rightarrow 40$ node-write-ops/sec (negligible)

$W+R = 6 > 5 \checkmark$ Strong consistency preserved

Overlap: $W+R-N = 1$ node minimum (still safe)

Even better: $W=5, R=1$ ($W+R=6 > 5 \checkmark$)

Every read hits 1 node $\rightarrow 100,000$ node-read-ops/sec (optimal for reads)

Every write hits all 5 nodes $\rightarrow 50$ node-write-ops/sec (still negligible)

Overlap: $W+R-N = 1 \checkmark$

BUT: $W=5$ means any node failure blocks all writes.

During AZ-B failure (2 nodes down): only 3 nodes alive, $W=5$ impossible

All writes fail during AZ failure.

AZ-aware recommendation: $W=4, R=2$

AZ-B failure: 3 nodes alive. Can still write to 3 of 5? No – need 4.

Reduce W at runtime to 3 during degraded mode (operator decision)

$R=2$ still works with 3 nodes $\checkmark$

Final answer: $W=4, R=2$ (normal) / $W=3, R=2$ (degraded/AZ-failure)

Use leader-based reads for true linearizability without quorum overhead.

Quick Reference: Consensus & IDs Cheat Sheet

Raft vs Paxos:

Both: quorum-based, leader-driven, safe under crash failures

Raft: explicit leader election, sequential log, easier to implement

Paxos: more flexible (multi-decree), used in Chubby/Spanner

Quorum formula:

Strong consistency: $W + R > N$

Max tolerable failures: $\min(N-W, N-R)$

Overlap guarantee: $W + R - N$ nodes

ID generation decision:

Need sortable + no coordination $\rightarrow$ UUID v7 or ULID

Need high throughput + small size $\rightarrow$ Snowflake

Need public-facing opaque ID $\rightarrow$ UUID v4

Service discovery:

Same language everywhere $\rightarrow$ Client-side (language SDK)

Polyglot / Kubernetes $\rightarrow$ Server-side (proxy / kube-proxy)

Both $\rightarrow$ Service mesh (Envoy sidecar = client-side without code)